

Mini-Project 2: Linear Regression, Variability, and Generalization

EN.553.434/634: Elements of Statistical Learning

Jonathan Ma

Setting things up

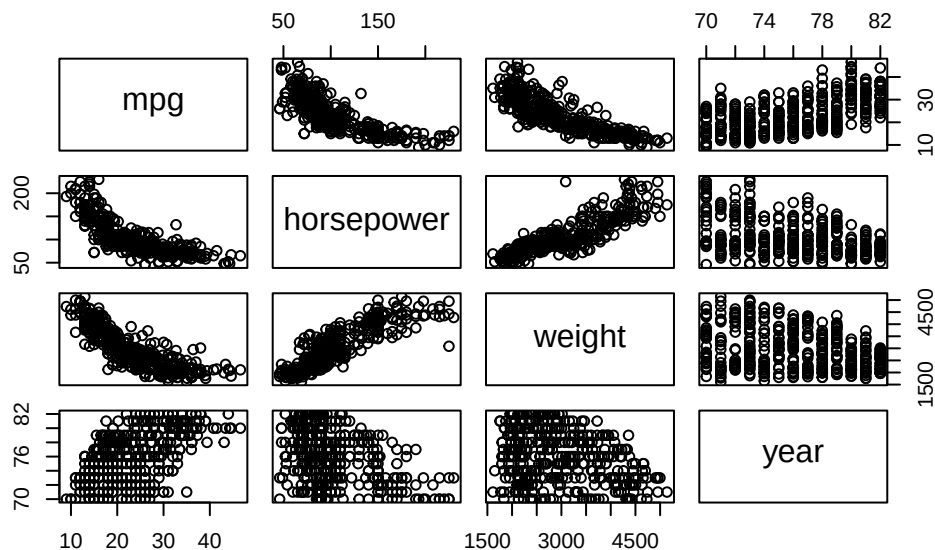
```
set.seed(434634)

auto <- read.csv("Auto.csv", na.strings = "?")

vars <- c("mpg", "horsepower", "weight", "year")

# Remove observations missing variables used in the project
auto <- auto[complete.cases(auto[vars]), ]

# Pairwise scatterplots and correlation matrix
pairs(auto[vars])
```



```
cor(auto[vars])
```

	mpg	horsepower	weight	year
mpg	1.0000000	-0.7784268	-0.8322442	0.5805410
horsepower	-0.7784268	1.0000000	0.8645377	-0.4163615
weight	-0.8322442	0.8645377	1.0000000	-0.3091199
year	0.5805410	-0.4163615	-0.3091199	1.0000000

```

# Mean squared error
mse <- function(y, yhat) {
  mean((y - yhat)^2)
}

# Design matrix for Parts 1--2
X <- cbind(
  1,
  auto$horsepower,
  auto$weight,
  auto$year
)
y <- auto$mpg

# 75/25 train-test split for Part 5
n <- nrow(auto)
train_id <- sample(seq_len(n), size = floor(0.75 * n))

auto_train <- auto[train_id, ]
auto_test  <- auto[-train_id, ]

```

1. Least squares on real data

We use

$$Y = \text{mpg}, \quad X_1 = \text{horsepower}, \quad X_2 = \text{weight}, \quad X_3 = \text{year}.$$

The sample size after removing observations with missing values is

```

n <- nrow(auto)
n

```

```
[1] 392
```

We first examine the relationship between mpg and each predictor.

```

par(mfrow = c(1, 3))

plot(
  auto$horsepower, auto$mpg,
  xlab = "Horsepower",
  ylab = "MPG",
  main = "MPG vs Horsepower"
)

plot(
  auto$weight, auto$mpg,
  xlab = "Weight",

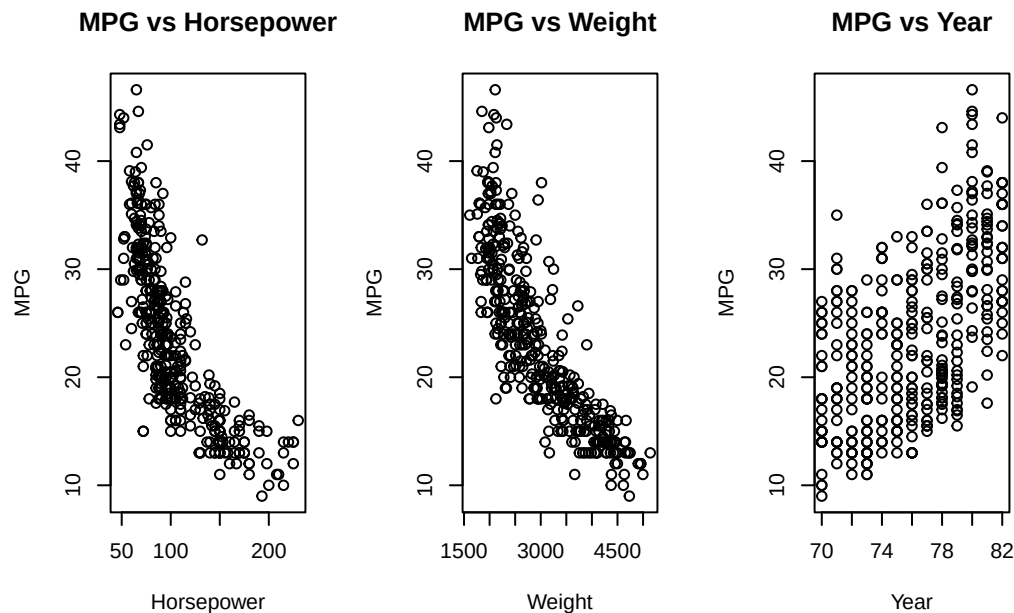
```

```

    ylab = "MPG",
    main = "MPG vs Weight"
)

plot(
  auto$year, auto$mpg,
  xlab = "Year",
  ylab = "MPG",
  main = "MPG vs Year"
)

```



```
par(mfrow = c(1, 1))
```

We fit the multiple linear regression model

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon.$$

The least-squares estimator is

$$\hat{\beta}_D = (X^T X)^{-1} X^T y.$$

```

# Compute OLS manually
beta_hat_manual <- solve(t(X) %*% X) %*% t(X) %*% y
beta_hat_manual

```

```

      [,1]
[1,] -13.719360033
[2,] -0.004999767
[3,] -0.006447746
[4,]  0.748705103

```

```
# Fit the same model using lm()
fit_auto <- lm(
  mpg ~ horsepower + weight + year,
  data = auto
)

coef(fit_auto)
```

```
(Intercept)    horsepower      weight      year
-13.719360033  -0.004999767  -0.006447746   0.748705103
```

```
# Compare the estimates
cbind(
  Manual = as.vector(beta_hat_manual),
  lm = coef(fit_auto)
)
```

```
          Manual      lm
(Intercept) -13.719360033 -13.719360033
horsepower  -0.004999767  -0.004999767
weight      -0.006447746  -0.006447746
year         0.748705103   0.748705103
```

Q1. Do the two methods give the same coefficient estimates, up to numerical precision? Interpret the estimated coefficient of **horsepower** in context.

Yes. The coefficient estimates obtained from the matrix formula and from `lm()` agree up to numerical precision. This is expected because both methods solve the same ordinary least-squares problem. The estimated coefficient of **horsepower** represents the expected change in **mpg** associated with a one-unit increase in horsepower, holding **weight** and **year** fixed. Since the estimated coefficient is negative, greater horsepower is associated with lower fuel economy after controlling for weight and model year.

2. Geometry of least squares

Using the fitted model from Part 1, we compute

$$\hat{y} = X\hat{\beta}_D, \quad e = y - \hat{y}.$$

```
# Fitted values and residuals
y_hat <- X %*% beta_hat_manual
e <- y - y_hat

# Orthogonality checks
Xt_e <- t(X) %*% e
sum_e <- sum(e)

Xt_e
```

```

      [,1]
[1,] 5.158611e-10
[2,] 5.373501e-08
[3,] 1.535944e-06
[4,] 3.912098e-08

```

```
sum_e
```

```
[1] 5.158611e-10
```

Q2. What does $X^T e \approx 0$ mean geometrically? Why does this imply that $\sum_i e_i \approx 0$?

The condition $X^T e \approx 0$ means that the residual vector e is orthogonal to every column of the design matrix X . Geometrically, $\hat{y}$ is the orthogonal projection of y onto $\text{Col}(X)$, so the remaining residual

$$e = y - \hat{y}$$

lies in the orthogonal complement of $\text{Col}(X)$. Because the first column of X is the intercept column $\mathbf{1}$, orthogonality also implies

$$\mathbf{1}^T e = \sum_{i=1}^n e_i \approx 0.$$

The computed quantities may differ slightly from zero because of floating-point numerical precision.

3. Sampling variability of $\hat{\beta}_D$

We now use a controlled simulation with $n = 100$ observations and three predictors,

$$X_1, X_2, X_3 \stackrel{\text{iid}}{\sim} N(0, 1).$$

We generate X once and hold it fixed throughout the experiment. The true coefficient vector and noise variance are

$$\beta^* = (1, 2, -1, 0.5)^T, \quad \sigma^2 = 1.$$

For each simulated training set,

$$y = X\beta^* + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2 I).$$

```

set.seed(434634)

n_sim <- 100
B <- 500
sigma2 <- 1
beta_star <- c(1, 2, -1, 0.5)

# Generate X once and hold it fixed

```

```

X_pred <- matrix(
  rnorm(n_sim * 3),
  nrow = n_sim,
  ncol = 3
)

X_sim <- cbind(1, X_pred)

# Store the coefficient estimates
beta_hats <- matrix(
  NA,
  nrow = B,
  ncol = 4
)

# Generate B independent response vectors using the same X
for (b in 1:B) {

  epsilon <- rnorm(
    n_sim,
    mean = 0,
    sd = sqrt(sigma2)
  )

  y_sim <- as.vector(
    X_sim %*% beta_star + epsilon
  )

  beta_hats[b, ] <-
    solve(t(X_sim) %*% X_sim) %*%
    t(X_sim) %*% y_sim
}

# Empirical summaries
empirical_mean <- colMeans(beta_hats)
empirical_sd <- apply(beta_hats, 2, sd)

# Theoretical covariance conditional on X
theoretical_cov <-
  sigma2 * solve(t(X_sim) %*% X_sim)

theoretical_sd <-
  sqrt(diag(theoretical_cov))

results_beta <- data.frame(
  Coefficient = c("Intercept", "X1", "X2", "X3"),
  True_Value = beta_star,

```

```

Empirical_Mean = empirical_mean,
Empirical_SD = empirical_sd,
Theoretical_SD = theoretical_sd
)

```

```
results_beta
```

	Coefficient	True_Value	Empirical_Mean	Empirical_SD	Theoretical_SD
1	Intercept	1.0	1.0014764	0.10257977	0.10076798
2	X1	2.0	2.0034216	0.09797294	0.10558834
3	X2	-1.0	-0.9973660	0.09243991	0.09353494
4	X3	0.5	0.4934706	0.10138633	0.10409219

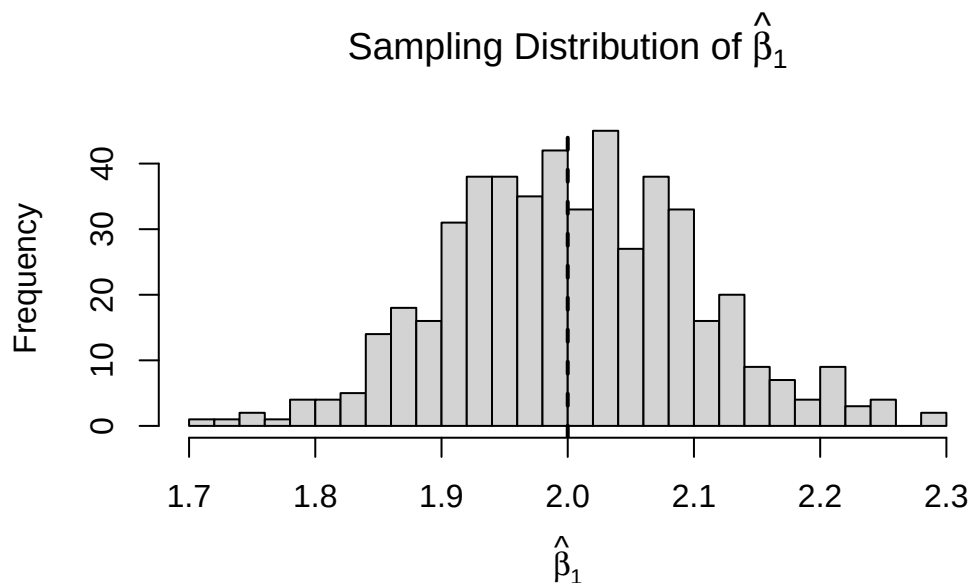
We can also visualize the sampling distribution of one of the slope estimates.

```

hist(
  beta_hats[, 2],
  breaks = 25,
  main = expression(
    "Sampling Distribution of " * hat(beta)[1]
  ),
  xlab = expression(hat(beta)[1])
)

abline(
  v = beta_star[2],
  lwd = 2,
  lty = 2
)

```



Q3. How do the empirical means compare with β^* ? How does this relate to $\mathbb{E}[\hat{\beta}_D | X] = \beta^*$?

The empirical means of the estimated coefficients are close to their corresponding true

values in

$$\beta^* = (1, 2, -1, 0.5)^T.$$

This agrees with the theoretical result

$$\mathbb{E}[\hat{\beta}_D | X] = \beta^*.$$

Because X is fixed and only the noise vector ε changes across training sets, repeated least-squares estimates fluctuate around the true coefficient vector. With only $B = 500$ simulations, the empirical means do not need to equal the true values exactly, but they should be close.

Q4. How do the empirical standard deviations compare with the theoretical standard deviations? Why is it important that X is held fixed in this experiment?

The empirical standard deviations are close to the theoretical standard deviations obtained from

$$\text{Var}(\hat{\beta}_D | X) = \sigma^2(X^T X)^{-1}.$$

Small differences are expected because the empirical standard deviations are estimated using only $B = 500$ simulated training sets. Holding X fixed is important because the theoretical covariance above is conditional on X . If we regenerated X for every training set, the observed variability in $\hat{\beta}_D$ would reflect randomness from both the predictor matrix and the response noise, so it would no longer directly correspond to the conditional variance formula for one fixed X .

4. Prediction uncertainty

Return to the Auto model from Part 1. Consider a car with

$$\text{horsepower} = 100, \quad \text{weight} = 3000, \quad \text{year} = 76.$$

The fitted mean response is

$$\hat{f}_D(x_0) = x_0^T \hat{\beta}_D.$$

```
# Predictor vector for the new car
x0 <- c(1, 100, 3000, 76)

# Fitted mean response
f_hat_x0 <-
  as.numeric(x0 %*% beta_hat_manual)

f_hat_x0

[1] 23.33901

# Estimate residual variance
p <- ncol(X)
```

```

sigma2_hat <-
  sum(e^2) / (nrow(X) - p)

# x0 leverage term
h0 <- as.numeric(
  t(x0) %*%
    solve(t(X) %*% X) %*%
    x0
)

# Standard error for the estimated mean
se_mean <-
  sqrt(sigma2_hat * h0)

# Standard error for a new observation
se_prediction <-
  sqrt(sigma2_hat * (1 + h0))

# 95% t critical value
t_crit <-
  qt(
    0.975,
    df = nrow(X) - p
  )

# Manual confidence interval
CI_mean <- c(
  lower = f_hat_x0 - t_crit * se_mean,
  fit = f_hat_x0,
  upper = f_hat_x0 + t_crit * se_mean
)

# Manual prediction interval
PI_new <- c(
  lower = f_hat_x0 - t_crit * se_prediction,
  fit = f_hat_x0,
  upper = f_hat_x0 + t_crit * se_prediction
)

CI_mean

```

	lower	fit	upper
CI_mean	22.98451	23.33901	23.69352

PI_new

	lower	fit	upper
PI_new	16.58535	23.33901	30.09268

We compare these calculations with the intervals returned by `lm()`.

```
new_car <- data.frame(  
  horsepower = 100,  
  weight = 3000,  
  year = 76  
)
```

```
predict(  
  fit_auto,  
  newdata = new_car,  
  interval = "confidence",  
  level = 0.95  
)
```

	fit	lwr	upr
1	23.33901	22.98451	23.69352

```
predict(  
  fit_auto,  
  newdata = new_car,  
  interval = "prediction",  
  level = 0.95  
)
```

	fit	lwr	upr
1	23.33901	16.58535	30.09268

Q5. Why is the prediction interval wider than the confidence interval? Identify the two sources of uncertainty represented in the prediction interval.

The prediction interval is wider because it accounts for two sources of uncertainty. First, there is uncertainty in estimating the conditional mean $f^*(x_0)$ from a finite training sample, represented by

$$\hat{\sigma}^2 x_0^T (X^T X)^{-1} x_0.$$

Second, a new individual response contains its own irreducible noise with variance $\hat{\sigma}^2$. Therefore, the prediction variance is

$$\hat{\sigma}^2 [1 + x_0^T (X^T X)^{-1} x_0].$$

The confidence interval only quantifies uncertainty about the mean response, while the prediction interval must additionally account for the random deviation of a new car's mpg from that mean.

5. Enlarging the linear model class

We now focus only on predicting mpg from horsepower. Using the 75/25 training-test split constructed above, we compare

$$\mathcal{F}_1 = \{\beta_0 + \beta_1 x\}$$

with

$$\mathcal{F}_2 = \{\beta_0 + \beta_1 x + \beta_2 x^2\}.$$

```
# Fit linear and quadratic models
fit_linear <- lm(
  mpg ~ horsepower,
  data = auto_train
)

fit_quadratic <- lm(
  mpg ~ horsepower + I(horsepower^2),
  data = auto_train
)

# Grid for plotting fitted curves
hp_grid <- seq(
  min(auto_train$horsepower),
  max(auto_train$horsepower),
  length.out = 300
)

grid_data <-
  data.frame(horsepower = hp_grid)

pred_linear_grid <-
  predict(fit_linear, newdata = grid_data)

pred_quad_grid <-
  predict(fit_quadratic, newdata = grid_data)

# Plot the training data
plot(
  auto_train$horsepower,
  auto_train$mpg,
  xlab = "Horsepower",
  ylab = "MPG",
  main = "Linear and Quadratic Fits"
)

lines(
  hp_grid,
  pred_linear_grid,
  lwd = 2
)

lines(
```

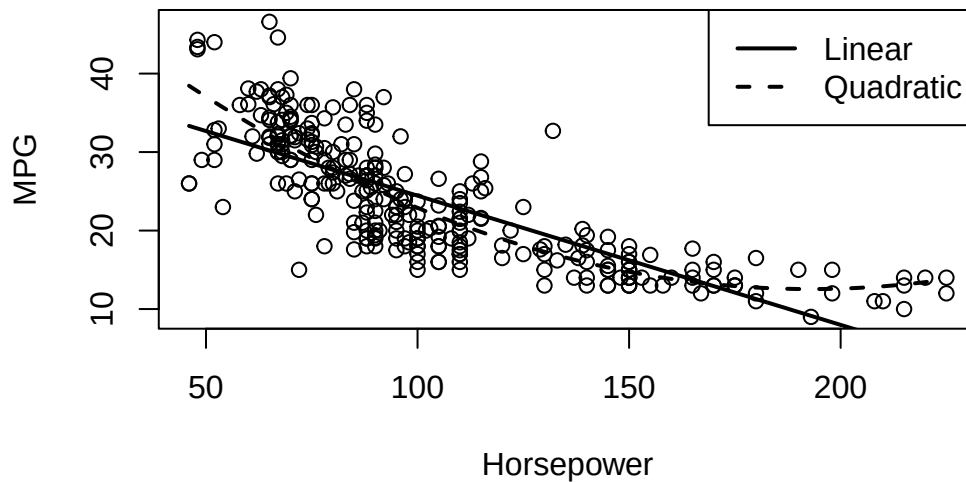
```

hp_grid,
pred_quad_grid,
lwd = 2,
lty = 2
)

legend(
  "topright",
  legend = c("Linear", "Quadratic"),
  lwd = 2,
  lty = c(1, 2)
)

```

Linear and Quadratic Fits



We next calculate the training and test MSE.

```

# Training predictions
train_pred_linear <-
  predict(fit_linear, newdata = auto_train)

train_pred_quad <-
  predict(fit_quadratic, newdata = auto_train)

# Test predictions
test_pred_linear <-
  predict(fit_linear, newdata = auto_test)

test_pred_quad <-
  predict(fit_quadratic, newdata = auto_test)

# MSE comparison
mse_results <- data.frame(
  Model = c("Linear", "Quadratic"),

```

```

Training_MSE = c(
  mse(auto_train$mpg, train_pred_linear),
  mse(auto_train$mpg, train_pred_quad)
),
Test_MSE = c(
  mse(auto_test$mpg, test_pred_linear),
  mse(auto_test$mpg, test_pred_quad)
)
)

mse_results

```

	Model	Training_MSE	Test_MSE
1	Linear	23.69453	25.24091
2	Quadratic	18.97503	19.40393

Q6. Why must the quadratic model have training error no larger than the linear model?

The quadratic model must have training error no larger than the linear model because the linear model class is nested inside the quadratic model class. Specifically,

$$\beta_0 + \beta_1 x + \beta_2 x^2$$

becomes a linear model when $\beta_2 = 0$. Therefore,

$$\mathcal{F}_1 \subseteq \mathcal{F}_2.$$

Least squares minimizes training error over the available model class, so minimizing over the larger class $\mathcal{F}_2$ cannot produce a larger minimum training error than minimizing over $\mathcal{F}_1$.

Q7. Compare the test errors of the two models. What does this example suggest about enlarging the model class?

The linear model has test MSE 25.241, while the quadratic model has test MSE 19.404. Thus, for this particular train-test split, the quadratic model has the lower test error. This illustrates that enlarging the model class always gives the model at least as much flexibility to fit the training data, but it does not guarantee better generalization. Whether the additional flexibility helps depends on whether the reduction in approximation error is sufficient to compensate for the additional estimation variability.

Q8. We do not know the true regression function f^* for the Auto data. Why can we not conclude from this experiment alone that the true relationship is linear or quadratic?

We cannot conclude that the true regression function is linear or quadratic because we observe only a finite, noisy sample from the underlying data-generating process. Test error tells us how these particular fitted models perform on held-out observations, not the exact form of the unknown regression function

$$f^*(x) = \mathbb{E}[Y \mid X = x].$$

In addition, we have compared only two candidate model classes. The true function could have some other nonlinear form, and the observed difference in test error is itself affected by the particular train-test split.

6. Dimension and generalization error

For each

$$d = 2, 5, 10, 20, 40,$$

we use $n = 400$ and generate

$$X \sim N(0, I_d), \quad Y = X^T \beta^* + \varepsilon, \quad \varepsilon \sim N(0, 1).$$

For each dimension,

$$\beta^* = \frac{1}{\sqrt{d}}(1, \dots, 1)^T.$$

This keeps the overall signal strength comparable as d changes.

```
set.seed(434634)

dims <- c(2, 5, 10, 20, 40)

n_train <- 400
n_test <- 5000
B_dim <- 100
sigma2_dim <- 1

dimension_results <- data.frame(
  d = dims,
  d_over_n = dims / n_train,
  Train_MSE = NA,
  Test_MSE = NA
)

for (j in seq_along(dims)) {

  d <- dims[j]

  beta_star_d <-
    rep(1 / sqrt(d), d)

  train_mse_b <-
    numeric(B_dim)

  test_mse_b <-
    numeric(B_dim)

  for (b in 1:B_dim) {
```

```

# Generate training data
X_train <- matrix(
  rnorm(n_train * d),
  nrow = n_train,
  ncol = d
)

epsilon_train <-
  rnorm(n_train)

y_train <- as.vector(
  X_train %*% beta_star_d +
  epsilon_train
)

# Fit least squares without an intercept
beta_hat <-
  solve(t(X_train) %*% X_train) %*%
  t(X_train) %*% y_train

# Training MSE
yhat_train <-
  X_train %*% beta_hat

train_mse_b[b] <-
  mse(y_train, yhat_train)

# Generate an independent test set
X_test <- matrix(
  rnorm(n_test * d),
  nrow = n_test,
  ncol = d
)

epsilon_test <-
  rnorm(n_test)

y_test <- as.vector(
  X_test %*% beta_star_d +
  epsilon_test
)

yhat_test <-
  X_test %*% beta_hat

test_mse_b[b] <-
  mse(y_test, yhat_test)

```

```

}

dimension_results$Train_MSE[j] <-
  mean(train_mse_b)

dimension_results$Test_MSE[j] <-
  mean(test_mse_b)
}

# Theoretical approximations
dimension_results$Train_Approx <-
  sigma2_dim *
  (1 - dimension_results$d_over_n)

dimension_results$Test_Approx <-
  sigma2_dim *
  (1 + dimension_results$d_over_n)

dimension_results

  d d_over_n Train_MSE Test_MSE Train_Approx Test_Approx
1  2   0.0050 0.9881578 1.003903         0.9950         1.0050
2  5   0.0125 0.9910450 1.015196         0.9875         1.0125
3 10   0.0250 0.9741075 1.025434         0.9750         1.0250
4 20   0.0500 0.9443462 1.055709         0.9500         1.0500
5 40   0.1000 0.8966831 1.114707         0.9000         1.1000

```

We compare the simulation results with

$$\mathbb{E}[\text{MSE}_{\text{train}}] \approx \sigma^2 \left(1 - \frac{d}{n}\right)$$

and

$$\mathbb{E}_D[R(\hat{f}_D)] \approx \sigma^2 \left(1 + \frac{d}{n}\right).$$

```

ylim_range <- range(
  dimension_results$Train_MSE,
  dimension_results$Test_MSE,
  dimension_results$Train_Approx,
  dimension_results$Test_Approx,
  sigma2_dim
)

plot(
  dimension_results$d_over_n,
  dimension_results$Train_MSE,

```

```

    type = "b",
    pch = 16,
    ylim = ylim_range,
    xlab = expression(d / n),
    ylab = "Mean Squared Error",
    main = "Dimension and Generalization Error"
)

lines(
  dimension_results$d_over_n,
  dimension_results$Test_MSE,
  type = "b",
  pch = 17,
  lty = 2
)

lines(
  dimension_results$d_over_n,
  dimension_results$Train_Approx,
  lty = 3
)

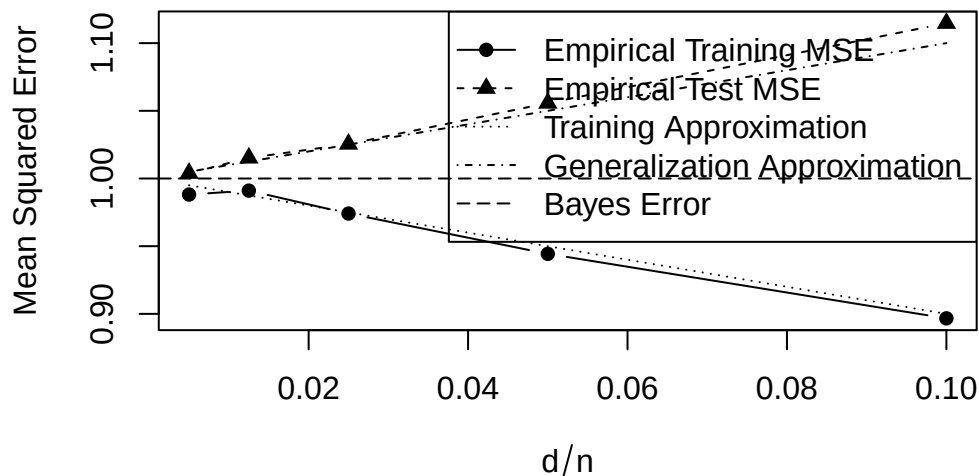
lines(
  dimension_results$d_over_n,
  dimension_results$Test_Approx,
  lty = 4
)

abline(
  h = sigma2_dim,
  lty = 5
)

legend(
  "topright",
  legend = c(
    "Empirical Training MSE",
    "Empirical Test MSE",
    "Training Approximation",
    "Generalization Approximation",
    "Bayes Error"
  ),
  lty = c(1, 2, 3, 4, 5),
  pch = c(16, 17, NA, NA, NA)
)

```

Dimension and Generalization Error



Q9. What happens to average training error and average generalization error as d/n increases?

As d/n increases, the average training error decreases while the average generalization error increases. This agrees with the approximations

$$\mathbb{E}[\text{MSE}_{\text{train}}] \approx \sigma^2 \left(1 - \frac{d}{n}\right)$$

and

$$\mathbb{E}_D[R(\hat{f}_D)] \approx \sigma^2 \left(1 + \frac{d}{n}\right).$$

Increasing the number of predictors gives least squares more freedom to fit the observed training data, including some of its random noise, which lowers training error. At the same time, more coefficients must be estimated from the same sample size, increasing estimation variability and therefore test error.

Q10. The true regression function belongs to the fitted linear model class for every value of d . Why does generalization error still increase with d ?

Correct specification eliminates approximation bias, but it does not eliminate estimation variance. Even though f^* belongs to the fitted linear model class for every d , the coefficients β^* must still be estimated from a finite sample. As d increases relative to n , more unknown coefficients are estimated from the same amount of data, so $\hat{\beta}_D$ becomes more variable across training sets. This additional estimation uncertainty increases generalization error. Approximately,

$$R(\hat{f}_D) - R(f^*) \approx \sigma^2 \frac{d}{n},$$

showing that the excess risk increases with the dimension-to-sample-size ratio.

Q11. Approximately how must n change if d doubles and we want to keep the excess generalization error approximately unchanged?

The approximate excess generalization error is

$$\sigma^2 \frac{d}{n}.$$

To keep this quantity approximately unchanged, the ratio d/n must remain constant. Therefore, if d doubles, n must also approximately double:

$$\frac{2d}{2n} = \frac{d}{n}.$$

Q12. Compare this result with the curse-of-dimensionality behavior you explored for kNN in Mini-Project 1.

Both experiments show that increasing dimension makes generalization more difficult, but the mechanisms and rates are different. For least squares under the correctly specified linear model, the excess generalization error grows approximately like d/n , so maintaining a similar level of estimation error requires n to grow roughly linearly with d . For kNN, increasing dimension causes neighborhoods to become increasingly sparse: observations that are close in one or a few dimensions may be far apart when many dimensions are considered. Maintaining the same local coverage can therefore require a sample size that grows much more rapidly with dimension. This is the stronger curse-of-dimensionality behavior associated with local nonparametric methods such as kNN.

7. Takeaways

Q13. In your own words, distinguish f^* , $f_{\mathcal{F}}^*$, and $\hat{f}_D$.

f^* is the optimal prediction function in the unrestricted population sense. Under squared-error loss,

$$f^*(x) = \mathbb{E}[Y \mid X = x].$$

$f_{\mathcal{F}}^*$ is the best population predictor available within a restricted model class $\mathcal{F}$:

$$f_{\mathcal{F}}^* = \arg \min_{f \in \mathcal{F}} R(f).$$

Finally, $\hat{f}_D$ is the function actually estimated from a finite training data set D by minimizing empirical risk over $\mathcal{F}$. Thus, f^* is the ideal population target, $f_{\mathcal{F}}^*$ is the best approximation to that target allowed by our chosen model class, and $\hat{f}_D$ is our data-dependent estimate.

Q14. What role does $(X^T X)^{-1}$ play in the variability of least-squares estimates?

Under the homoskedastic linear model,

$$\text{Var}(\hat{\beta}_D \mid X) = \sigma^2 (X^T X)^{-1}.$$

Therefore, $(X^T X)^{-1}$ determines how the geometry of the predictors translates response noise into variability of the coefficient estimates. Directions in the design matrix that contain little information, such as directions associated with highly correlated predictors or small eigenvalues of $X^T X$, correspond to greater estimation variance.

Q15. Why can enlarging a model class improve training fit but worsen generalization?

Enlarging the model class gives the fitting procedure more possible functions to choose from, so the minimum achievable training error cannot increase. However, the additional flexibility can also make the fitted function more sensitive to random features of the particular training sample. Enlarging the model class can therefore reduce approximation error while increasing estimation variance. If the increase in variance outweighs the reduction in approximation error, generalization performance can worsen even though training fit improves.

Generative AI use

State whether you used generative AI. If yes, briefly describe what you used it for.

GAI was used to add informative comments and fix some equation formatting